

Title: UPI Payment and Digital Wallet System using Interface and Exception Handling

UpiAndDigital.java

Objective / Aim

To design and implement a in Java using , simulating real-world digital payment operations such as wallet creation, money addition, UPI payments, balance checking, and transaction validation.

Problem Statement

Design a for a user.
The system should allow a user to:

Create a wallet with a name, mobile number, and UPI ID.
Add money to the wallet.
Make a payment using the UPI ID of another user.
Check the available wallet balance.
Validate the UPI ID before processing a payment.
Prevent payments when the balance is insufficient.
Prevent invalid or negative transaction amounts.
Handle invalid transactions using appropriate exceptions.
Display the final wallet details and transaction status.

The program must demonstrate:

Class and Interface Design

The following components should be created:

1. PaymentService Interface

Define the common operations:

```
pay()  
checkBalance()
```

2. Wallet Class

Store:
User name
Mobile number

UPI ID

Balance

The data members should be declared to demonstrate encapsulation.

Methods:

addMoney()

getBalance()

displayWalletDetails()

3. UPIPayment Class

Implement the PaymentService interface.

Methods:

pay()

checkBalance()

4. Custom Exceptions

Create:

InsufficientBalanceException

InvalidUPIException

InvalidAmountException

Steps to do the task:

Create an interface named PaymentService.

Declare pay() and checkBalance() methods in the interface.

Create a Wallet class.

Declare user information and balance as private data members.

Create a constructor to initialize the wallet details.

Define addMoney() to add money to the wallet.

Create an UPIPayment class that implements PaymentService.

Create an object of Wallet and associate it with the UPIPayment object.

Validate the UPI ID using String operations.

If the UPI ID is invalid, throw InvalidUPIException.

Validate the payment amount.

If the amount is zero or negative, throw InvalidAmountException.

Check whether sufficient balance is available.

If the balance is insufficient, throw InsufficientBalanceException.

If all validations are successful, deduct the payment amount from the wallet balance.

Display the payment success message.

Use try-catch-finally to handle transaction exceptions.

Display the final wallet balance.

✂ System Requirements

Hardware

- Standard PC/Laptop

- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

This program uses **custom (user-defined) exceptions** — your own exception classes created by extending `Exception`. They let you report business-rule errors with clear, meaningful names instead of generic ones.

Exception	When it is thrown	Type
<code>InsufficientBalanceException</code>	Wallet balance is less than the amount being sent	Custom, Checked
<code>InvalidUPIException</code>	Receiver UPI ID is empty or has no @ symbol	Custom, Checked
<code>InvalidAmountException</code>	Amount entered is zero or negative	Custom, Checked
<code>NumberFormatException</code>	User types letters where a number is expected	Built-in, Unchecked

All three custom exceptions **extend `Exception`**, which makes them **checked exceptions** — so any method that throws them must declare `throws`, and the caller must catch them.

The program also uses a **multi-catch block** in `UPIPayment.pay()`:

```
catch (InvalidUPIException | InvalidAmountException |
InsufficientBalanceException e)
```

This handles three different exception types in one block using the `|` (pipe) symbol.

2. Step-wise Execution

PHASE 1 – REGISTRATION (loop until all details are valid)

1. Scanner object is created.
2. A while loop runs while `valid == false`:
 - a. Read name -> IF empty, print error and restart loop.
 - b. Read phone -> IF length `!= 10`, print error and restart loop.
-> Try `Long.parseLong()`; if `NumberFormatException`, print error and restart loop.
 - c. Read UPI ID -> IF empty or missing '@', print error, restart loop.

- d. Read balance -> Try Long.parseLong(); if NumberFormatException, print error and restart loop.
 - > IF balance <= 0, print error and restart loop.
- e. All checks passed -> set valid = true, exit loop.

PHASE 2 – WALLET CREATION

- 3. Wallet object 'myWallet' is created with name, phone, upi, balance.

PHASE 3 – DISPLAY DETAILS (optional)

- 4. Ask "see wallet details? (yes/no)".
 - IF yes -> displayWalletDetails() prints name, phone, UPI, balance.

PHASE 4 – ADD MONEY (optional)

- 5. Ask "add money? (yes/no)".
 - IF yes -> read amount.
 - > Long.parseLong() may throw NumberFormatException -> caught.
 - > addmoney(amount) may throw InvalidAmountException (if amount <= 0) -> caught, message printed.
 - > ELSE balance increases, new balance printed.

PHASE 5 – SEND MONEY (optional)

- 6. Ask "send money? (yes/no)".
 - IF yes -> read receiver UPI and amount.
 - > Long.parseLong() may throw NumberFormatException -> caught.
 - > UPIPayment object is created (implements PaymentService).
 - > payment.pay() calls wallet.payMoney(), which checks in order:
 - i. UPI empty -> InvalidUPIException
 - ii. UPI has no '@' -> InvalidUPIException
 - iii. amount <= 0 -> InvalidAmountException
 - iv. balance < amount -> InsufficientBalanceException
 - v. All OK -> balance reduced, success message printed.
 - > Any of these three exceptions is caught by the MULTI-CATCH block, printing "Payment failed: <message>".
 - > payment.checkBalance() prints current balance.

PHASE 6 – FINISH

- 7. Final balance is printed.
- 8. "Program finished." is printed.
- 9. Scanner is closed.

3. Algorithm

START

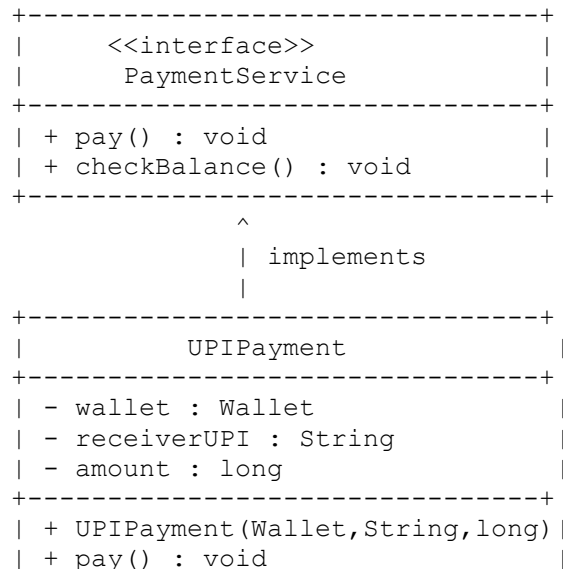
- 1. Create Scanner object
- 2. SET valid = false
- 3. WHILE valid is false:
 - a. Read name; IF empty, print error and CONTINUE
 - b. Read phone; IF length != 10, print error and CONTINUE
 - TRY parse phone to long
 - CATCH NumberFormatException -> print error, CONTINUE
 - c. Read UPI; IF empty OR does not contain '@', print error and CONTINUE
 - d. Read balance
 - TRY parse balance to long
 - CATCH NumberFormatException -> print error, CONTINUE

```

        IF balance <= 0, print error and CONTINUE
    e. SET valid = true
4. Create Wallet object with (name, phone, upi, balance)
5. ASK "see wallet details?"
    IF yes -> call displayWalletDetails()
6. ASK "add money?"
    IF yes -> read amount
    TRY:
        parse amount, call addmoney(amount)
        IF amount <= 0, THROW InvalidAmountException
        ELSE balance = balance + amount, print new balance
    CATCH NumberFormatException -> print "valid number" message
    CATCH InvalidAmountException -> print exception message
7. ASK "send money?"
    IF yes -> read receiverUPI and amount
    TRY:
        parse amount
        Create UPIPayment(wallet, receiverUPI, amount)
        Call pay():
            TRY wallet.payMoney(receiverUPI, amount)
                IF UPI empty -> THROW InvalidUPIException
                IF UPI has no '@' -> THROW InvalidUPIException
                IF amount <= 0 -> THROW InvalidAmountException
                IF balance < amount -> THROW InsufficientBalanceException
                ELSE balance = balance - amount, print success
            CATCH (InvalidUPIException | InvalidAmountException |
                InsufficientBalanceException) -> print "Payment failed"
        Call checkBalance() -> print current balance
    CATCH NumberFormatException -> print "valid number" message
8. Print final balance
9. Print "Program finished."
10. Close Scanner
STOP

```

4. UML Class Diagram



Code:

```
package oops;

import java.util.Scanner;

// 4. Custom Exceptions

//these are checked exceptions
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

class InvalidUPIException extends Exception {
    public InvalidUPIException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

//1.paymentservice interface
interface PaymentService {
    void pay();
    void checkBalance();
}

//2.wallet class
class Wallet {
    private String User;
    private long phno;
    private String UPI_ID;
```

```
private long Balance;

//this is a constructor to create wallet object
public Wallet(String User, long phno, String UPI_ID, long
Balance) {
    this.User = User;
    this.phno = phno;
    this.UPI_ID = UPI_ID;
    this.Balance = Balance;
}

//this method adds money to wallet
public void addmoney(long amount) throws
InvalidAmountException {
    if (amount <= 0) {
        throw new InvalidAmountException("Amount must be
greater than zero.");
    }
    Balance = Balance + amount;
    System.out.println("Money added successfully. New
balance: " + Balance);
}

public long getBalance() {
    return Balance;
}

//this method displays all wallet details
public void displayWalletDetails() {
    System.out.println("User Name: " + User);
    System.out.println("Phone Number: " + phno);
    System.out.println("UPI ID: " + UPI_ID);
    System.out.println("Balance: " + Balance);
}

public void payMoney(String receiverUPI, long amount)
```

```
        throws InvalidUPIException, InvalidAmountException,
InsufficientBalanceException {

    // Check the UPI ID using String operations.
    if (receiverUPI == null ||
receiverUPI.trim().isEmpty()) {
        throw new InvalidUPIException("UPI ID cannot be
empty.");
    }

    if (!receiverUPI.contains("@")
        || receiverUPI.startsWith("@")
        || receiverUPI.endsWith("@")
        || receiverUPI.indexOf("@") !=
receiverUPI.lastIndexOf("@")) {
        throw new InvalidUPIException("UPI ID is not
valid.");
    }

    // The payment amount must be positive.
    if (amount <= 0) {
        throw new InvalidAmountException("Amount must be
greater than zero.");
    }

    //check whether wallet has enough balance
    if (Balance < amount) {
        throw new
InsufficientBalanceException("Insufficient balance in
wallet.");
    }

    // Deduct money only after all validations are
successful.
    Balance = Balance - amount;
```

```

        System.out.println("Payment sent to " + receiverUPI + "
for " + amount + ".");
    }
}
//upipayment class
class UPIPayment implements PaymentService {
    private Wallet wallet;
    private String receiverUPI;
    private long amount;

    //wallet object is connected with payment object here
    public UPIPayment(Wallet wallet, String receiverUPI, long
amount) {
        this.wallet = wallet;
        this.receiverUPI = receiverUPI;
        this.amount = amount;
    }

    @Override
    public void pay() {
        //try block runs the payment
        try {
            wallet.payMoney(receiverUPI, amount);
            //catch block handles payment errors
        } catch (InvalidUPIException | InvalidAmountException |
InsufficientBalanceException e) {
            System.out.println("Payment failed: " +
e.getMessage());
        } finally {
            // This block runs whether payment succeeds or
fails.
            System.out.println("Transaction completed.");
        }
    }

    @Override

```

```

        public void checkBalance() {
            System.out.println("Current Balance: " +
wallet.getBalance());
        }
    }

public class UpiAndDigital {
    public static void main(String[] args) {
        //Scanner takes input from user
        Scanner sc = new Scanner(System.in);

        String name="" ;
        long phone=0 ;
        String upi="" ;
        long balance =0;
        boolean choice = false;

        // Keep checking until all user details are correct.
        while (!choice) {
            System.out.println("Enter your name:");
            name = sc.nextLine();

            if (name == null || name.trim().isEmpty()) {
                System.out.println("Name cannot be empty.");
                continue;
            }

            System.out.println("Enter your phone number:");
            String phoneInput = sc.nextLine();

            //phone number must have 10 digits
            if (phoneInput.length() != 10) {
                System.out.println("Phone number must have 10
digits.");
                continue;
            }

```

```
        try {
            phone = Long.parseLong(phoneInput);
        } catch (NumberFormatException e) {
            System.out.println("Phone number must contain
only numbers.");
            continue;
        }

        System.out.println("Enter your UPI ID:");
        upi = sc.nextLine();

        //UPI ID must contain @ symbol
        if (upi == null || upi.trim().isEmpty() ||
!upi.contains("@")) {
            System.out.println("UPI ID is invalid. It must
contain '@'.");
            continue;
        }

        System.out.println("Enter your starting balance:");
        String balanceInput = sc.nextLine();

        try {
            balance = Long.parseLong(balanceInput);
        } catch (NumberFormatException e) {
            System.out.println("Balance must be a
number.");
            continue;
        }

        if (balance <= 0) {
            System.out.println("Balance must be greater
than zero.");
            continue;
        }
    }
}
```

```
        choice = true;
    }

    // Only after all values are correct, create the wallet
    object.
    Wallet myWallet = new Wallet(name, phone, upi,
    balance);

    System.out.println("Do you want to see wallet details?
    (yes/no)");
    String detailChoice = sc.nextLine();

    if (detailChoice.equalsIgnoreCase("yes")) {
        myWallet.displayWalletDetails();
    }

    System.out.println("Do you want to add money?
    (yes/no)");
    String addChoice = sc.nextLine();

    if (addChoice.equalsIgnoreCase("yes")) {
        System.out.println("Enter amount to add:");
        String addInput = sc.nextLine();

        try {
            long addAmount = Long.parseLong(addInput);
            myWallet.addmoney(addAmount);
        } catch (NumberFormatException e) {
            System.out.println("Please enter a valid
            number.");
        } catch (InvalidAmountException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```
        // Payment is done only if user says yes.
        System.out.println("Do you want to send money?
(yes/no)");
        String sendChoice = sc.nextLine();

        if (sendChoice.equalsIgnoreCase("yes")) {
            System.out.println("Enter receiver UPI ID:");
            String receiverUPI = sc.nextLine();

            System.out.println("Enter amount to send:");
            String sendAmountInput = sc.nextLine();

            try {
                long sendAmount =
Long.parseLong(sendAmountInput);
                //create payment object and pass wallet object
to it
                UPIPayment payment = new UPIPayment(myWallet,
receiverUPI, sendAmount);
                payment.pay();
                payment.checkBalance();
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid
number.");
            }
        }

        System.out.println("Final balance: " +
myWallet.getBalance());
        System.out.println("Program finished.");
        sc.close();
    }
}
```

Output Screenshot:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'oops.UpiAndDigital'
Enter your name:
susmitha
Enter your phone number:
923121444
Phone number must have 10 digits.
Enter your name:
susmitha
Enter your phone number:
9876543210
Enter your UPI ID:
qwerty@122
Enter your starting balance:
50000
Do you want to see wallet details? (yes/no)
yes
User Name: susmitha
Phone Number: 9876543210
UPI ID: qwerty@122
Balance: 50000
Do you want to add money? (yes/no)
yes
Enter amount to add:
2350000
Money added successfully. New balance: 2400000
Do you want to send money? (yes/no)
yes
Enter receiver UPI ID:
2432aasda@23
Enter amount to send:
2111
Payment sent to 2432aasda@23 for 2111.
Current Balance: 2397889
Final balance: 2397889
Program finished.
```

```
File Edit Selection View Go Run Terminal Help
basicjava
Problems 12 Output Debug Console Terminal Ports
Enter your name:
1qewertyu
Enter your phone number:
1234455555
Enter your UPI ID:
123we@
Enter your starting balance:
0
Balance must be greater than zero.
Enter your name:
qwerty
Enter your phone number:
1234567890
Enter your UPI ID:
122324
UPI ID is invalid. It must contain '@'.
Enter your name:
qwerty
Enter your phone number:
1234567890
Enter your UPI ID:
1qwe@3wt
Enter your starting balance:
2094794
Do you want to see wallet details? (yes/no)
no
Do you want to add money? (yes/no)
no
Do you want to send money? (yes/no)
yes
Enter receiver UPI ID:
qweret@132
Enter amount to send:
qwere
Please enter a valid number.
Final balance: 2094794
Program finished.
```

Key OOP concepts shown:

- **Abstraction** — `PaymentService` is an interface declaring *what* a payment class must do, not *how*.
- **Implementation** — `UPIPayment` implements `PaymentService` and overrides both methods.

- **Encapsulation** — `Wallet` keeps all fields `private`, exposing them only through methods.
- **Association (has-a)** — `UPIPayment` holds a `Wallet` object rather than inheriting from it.
- **Custom exceptions** — three classes extend `Exception` to signal specific business errors.
- **Multi-catch** — one catch block handles three exception types using `|`.